МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ
імені ІГОРЯ СІКОРСЬКОГО»

Звіт з практичної роботи №2

з дисципліни «Розробка програмного забезпечення мобільних пристроїв»

Практичні заняття №5–8

«Меню, форми, MVVM, локальне сховище даних»

Варіант №1 — «Система прогнозування споживання електроенергії»

Просунутий рівень

Виконала студентка групи ТВ-з31
Вахрушева Христина Костянтинівна

Перевірив
Олексій Леонідович Недашківський

Київ — 2026

1 Мета, завдання та джерела

Мета роботи — реалізувати Android-застосунок для оцінювання споживання електроенергії з багатоекранною навігацією, інтерактивними формами, архітектурою MVVM, реактивним станом і локальним збереженням даних. Практична робота №2 охоплює заняття №5–8 методичних вказівок [1].

Індивідуальний варіант №1 — «Система прогнозування споживання електроенергії». Наведені у посібнику приклади сонячної електростанції належать варіанту №30 і використані лише як демонстрація підходів, а не як предметна область цього застосунку. Реалізовано рівень складності, орієнтований на критерії «просунутий» для всіх чотирьох занять.

Роботу описано на основі методичних вказівок [1], вихідного коду [2], скриншотів інтерфейсу та підсумків перевірок Gradle. Скриншоти демонструють окремі стани застосунку; результати автоматизованих перевірок наведено окремо в розділі 9. Відповідність вимогам узагальнено в табл. 1.

Версія коду: feb433757ed9896859ba673c6d69fca10fec9021. Каталог проєкту: https://github.com/Christ1naa/RPZMP/tree/feb433757ed9896859ba673c6d69fca10fec9021/PW02. Перевірки виконано локально; незмінне посилання на перевірену версію наведено у джерелі [2].

1.1 Відповідність критеріям занять №5–8

Таблиця 1 — Матриця відповідності вимогам методичних вказівок

Вимога і сторінка [1]	**Реалізація та перевірювані файли**	**Доказ / статус**
№5: меню, NavHost, аргументи, оновлення UI (с. 13–14)	ui/navigation/AppNavHost.kt, AppRoutes.kt; ui/screens/MainMenuScreen.kt	Код підтверджено; рис. 1–2
№5: компоненти, адаптивність, коментарі (с. 14)	ui/screens/*; ui/components/*; KDoc	Адаптивність — рис. 11; українські пояснювальні коментарі до всіх 32 іменованих composable-функцій, включно з Preview
№5: вхід і реєстрація (с. 14–15)	ui/screens/AuthScreen.kt; ui/auth/AuthViewModel.kt	Форми — рис. 3; валідація — рис. 4; unit-тести — рис. 12
№6: поля, підказки, комплексна валідація, loading (с. 25–26)	AuthScreen.kt; DeviceEditorDialog.kt; logic/*Validator.kt	Валідація — рис. 4–5; список — рис. 6; логіка станів — AuthViewModel.kt
№7: ViewModel, StateFlow, корутини, стани (с. 35–37)	ForecastViewModel.kt; ForecastHistoryViewModel.kt; AppNavHost.kt	Стан і корутини — ForecastViewModel.kt; результат — рис. 7
№8: Repository, кілька джерел, кеш, помилки (с. 46–47)	RoomEnergyRepository.kt; data/source/*	Підтверджено кодом; рис. 8
№8: таблиці, зв’язки, транзакції, обчислення (с. 47–48)	EnergyDao.observeProfile(); RoomEnergyRepository; ForecastViewModel; EnergyCalculator	Зв’язки — табл. 3; транзакції та SUM/COALESCE — EnergyDao; SQL-підсумок використовується в UI; тести — рис. 12–13
№8: реактивний список, пошук, фільтрація, сортування (с. 47–48)	ForecastUiState.visibleDevices; ForecastHistoryUiState.visibleForecasts	Список — рис. 6; історія — рис. 9–10; UI-фільтрація у Kotlin

Критерії просунутого рівня заняття №8 реалізовано через пов’язані таблиці, транзакції та робоче обчислюване поле SQL-запиту. EnergyDao.observeProfile() повертає добове споживання dailyConsumptionKwh як суму енергії всіх приладів поточного ownerKey. RoomEnergyRepository передає значення до SavedEnergyProfile, ForecastViewModel — до ForecastUiState і картки прогнозу. Обробку цього значення та реактивні оновлення перевіряють тести, наведені в розділі 9.

2 Засоби розроблення та структура проєкту

Роботу виконано для Android із використанням Kotlin, Jetpack Compose, Material 3, Navigation Compose, ViewModel, StateFlow, Kotlin Coroutines та Room. Розроблення і локальна перевірка здійснювалися в Windows з Android Studio та PowerShell; ручні й інструментальні перевірки проводилися на емуляторі Pixel_6 (Android 16). Параметри конфігурації наведено в табл. 2; призначення засобів розроблення розкрито в документації Android та Kotlin [3, 4].

Таблиця 2 — Параметри конфігурації, підтверджені репозиторієм

Параметр	**Значення**
Application ID / namespace	ua.kpi.energyforecast.pw02 / ua.kpi.energyforecast
Android Gradle Plugin / Gradle Wrapper	9.4.1 / 9.6.0
Kotlin Compose plugin / KSP	2.2.10 / 2.3.10
Compose BOM / Navigation Compose	2026.02.01 / 2.9.8
Room / Lifecycle ViewModel	2.8.4 / 2.10.0
compileSdk / targetSdk / minSdk	37 / 37 / 26
Java source / target compatibility	11 / 11
База даних	energy_forecast.db, версія 2

Головні вихідні пакети: logic (валідація та розрахунки), model (моделі), data/local (Entity, DAO, Room), data/source (JSON-джерело й кеш), data (репозиторії), ui/navigation (маршрути), ui/forecast та ui/history (ViewModel і стани), ui/screens (екрани) і ui/components (повторювані елементи). Розташування файлів відповідає фактичному дереву PW02/app/src/main/java/ua/kpi/energyforecast.

3 Головне меню, навігація та авторизація

Головне меню надає переходи до списку приладів, швидкого прогнозу, демонстраційного екрана та історії. AppNavHost визначає маршрути, а NavController виконує переходи. Для швидкого прогнозу передається аргумент днів типу Int, який отримує QuickForecastViewModel через SavedStateHandle. Для деталей історії передається ID прогнозу, за яким запис вибирається з повного списку історії. Повернення здійснюється popBackStack().

Підсумок останнього швидкого прогнозу зберігається вище NavHost у MainScreen і відображається після повернення до меню. На рис. 2 показано розрахунок за 7 днів і оновлену картку меню з результатом 14,00 кВт·год. Відкритий прогноз у деталях шукається в повному historyState.forecasts, тому зміна пошуку чи фільтра історії не приховує його. Головне меню наведено на рис. 1.

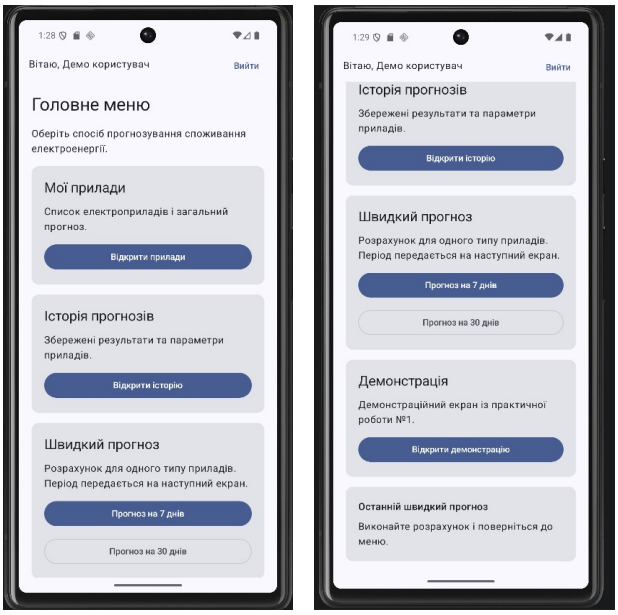


Рисунок 1 — Головне меню застосунку EnergyForecast PW02

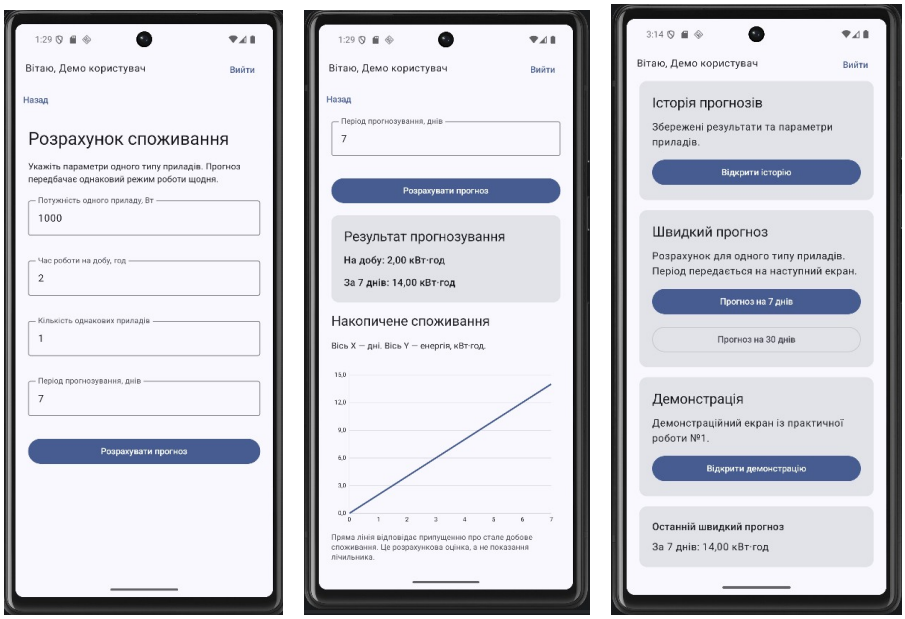


Рисунок 2 — Швидкий прогноз і повернення до меню

Навчальна авторизація й реєстрація мають перевірку введення, повідомлення про помилки, стани завантаження та запобігання повторному відправленню. Реєстраційні акаунти перебувають у пам’яті демонстраційного репозиторію, а не в Room. Після завершення процесу вони не відновлюються. Тому доступ за email не є захищеною серверною автентифікацією. Форми входу й реєстрації наведено на рис. 3, приклади повідомлень валідації — на рис. 4. Для реєстрації ім’я після видалення крайових пробілів має містити 2–60 символів, пароль — 8–64 символи, щонайменше одну літеру й одну цифру. Підтвердження має точно збігатися з паролем. Перевіряються формат email, його довжина до 254 символів і відсутність акаунта з таким email у демонстраційному репозиторії.

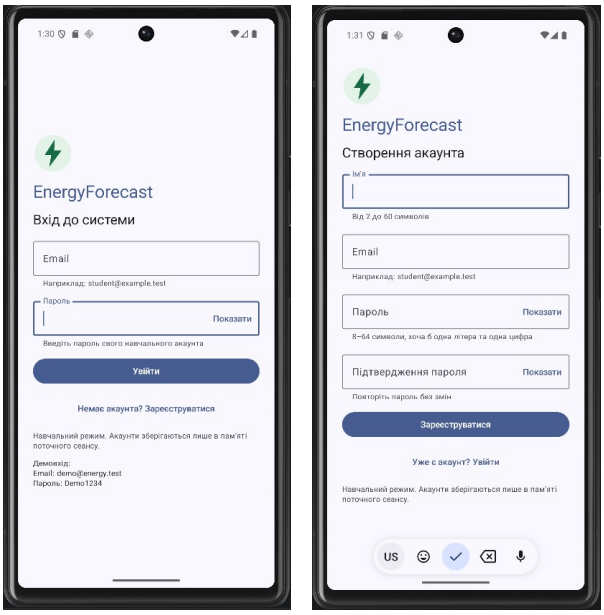


Рисунок 3 — Форма входу та реєстрації

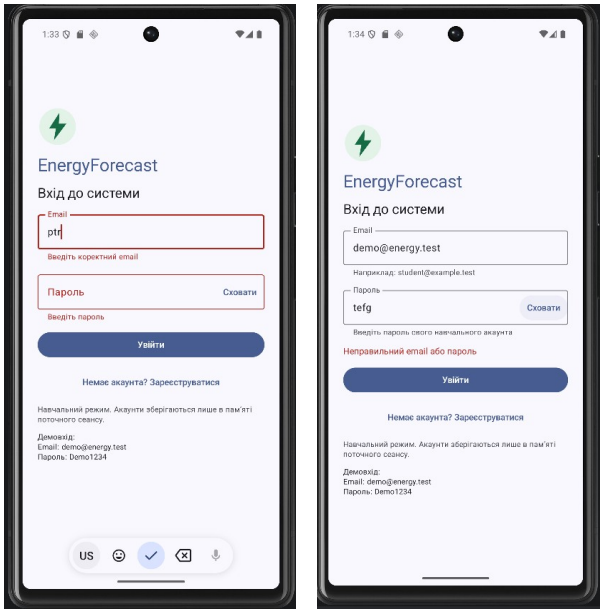


Рисунок 4 — Повідомлення валідації форми

4 Форми введення та керування приладами

На екрані приладів користувач переглядає список, додає, змінює та видаляє позиції. Назва після trim має містити 1–60 символів; потужність — скінченне число $0 < P \le 100000$ Вт; час роботи — 0–24 години на добу; кількість — ціле число 1–1000. Період прогнозу обмежений 1–365 днями. RoomEnergyRepository обмежує профіль 50 позиціями. Некоректні значення не записуються до сховища.

Список має пошук і сортування за назвою або розрахунковим добовим споживанням. Властивість ForecastUiState.visibleDevices виконує ці операції у Kotlin з підтримкою українського порядку літер. Вони змінюють лише відображення списку, тому пошуковий запит не впливає на загальний прогноз. Добовий підсумок повного профілю обчислюється в SQL. Форму редагування та список наведено на рис. 5–6.

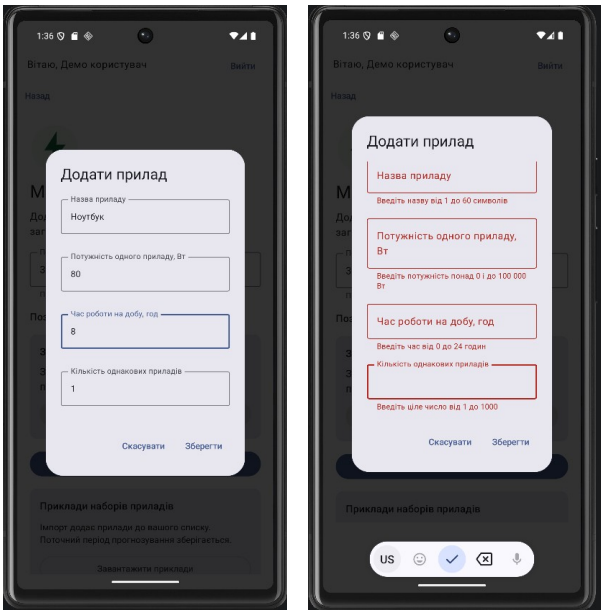


Рисунок 5 — Форма додавання та перевірка параметрів приладу

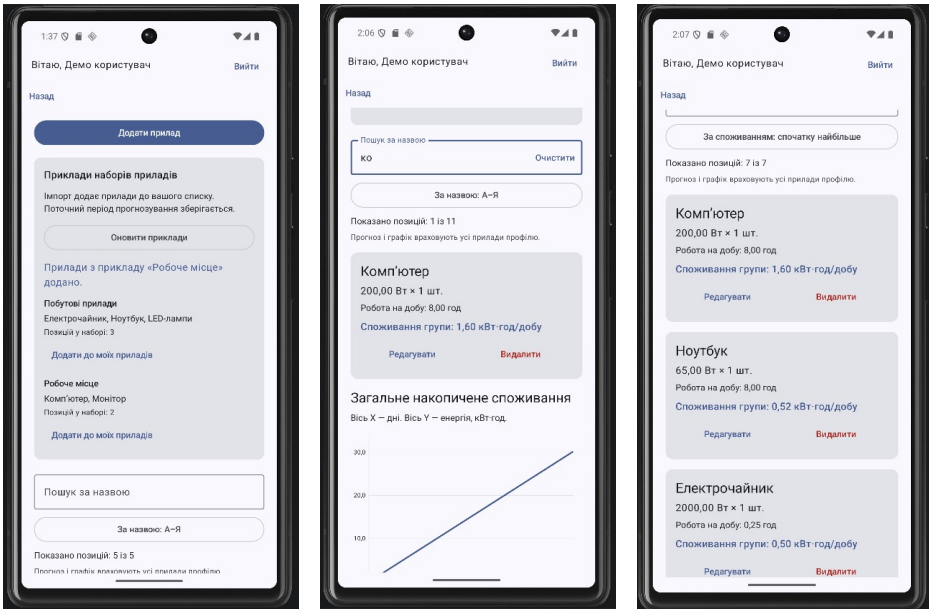


Рисунок 6 — Список приладів із пошуком та сортуванням

5 Архітектура MVVM та реактивний стан

UI (Compose) → ViewModel → Repository → Room / JSON

ForecastViewModel одержує поточний профіль і добовий SQL-підсумок через Repository.observeProfile() та надає ForecastUiState як StateFlow. Після змін у Room потік Flow доставляє узгоджені список і підсумок; collectAsStateWithLifecycle забезпечує оновлення Compose-інтерфейсу. Корутини виконуються у viewModelScope. Через withContext(calculationDispatcher), де типовим є Dispatchers.Default, ViewModel викликає repository.calculateForecastFromDaily(): значення перевіряється й множиться на вибраний період. Застаріле обчислення скасовується, щоб показувати результат актуального введення.

Записи до профілю ForecastViewModel ставить у Channel, щоб зберігати порядок дій користувача. Помилки читання, запису та розрахунку відображаються окремими повідомленнями. ForecastHistoryViewModel керує історією, пошуком, фільтрами та станами isSaving/isDeleting; повторні команди збереження або видалення блокуються під час відповідної операції. Це демонструє передбачені заняттям №7 стани завантаження, успіху і помилки.

Стислий приклад із фактичного ForecastViewModel.kt:

```
private val mutableUiState = MutableStateFlow(
    ForecastUiState(isLoading = true)
)
val uiState: StateFlow<ForecastUiState> = mutableUiState.asStateFlow()
```

Рисунок 7 демонструє результат контрольного розрахунку та частину списку приладів. Стани завантаження й помилки реалізовано у ViewModel і відповідних компонентах інтерфейсу; статичний кадр із готовим результатом не відображає перебіг асинхронної операції. KDoc у ключових класах пояснює взаємодію ViewModel, сховища й кешу.

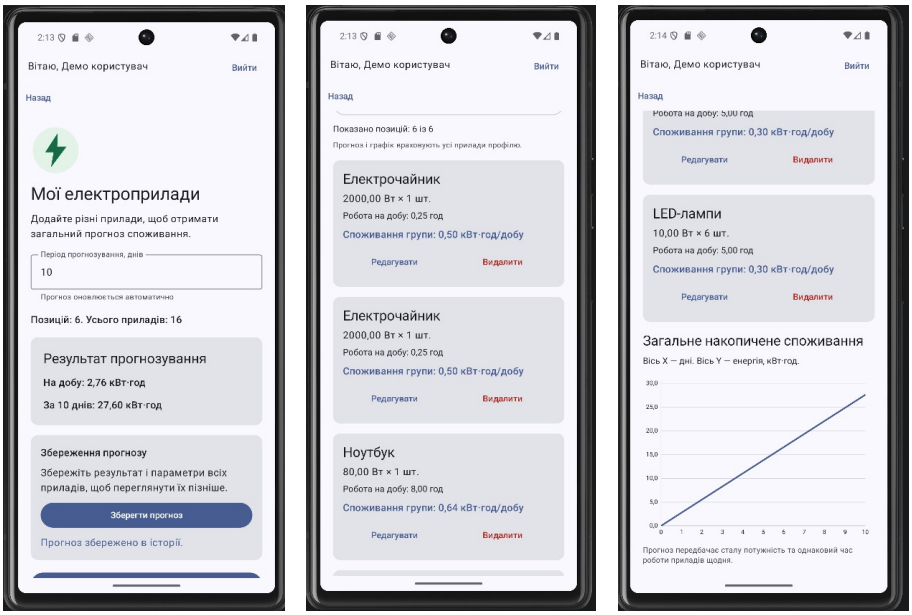


Рисунок 7 — Контрольний прогноз: 2,76 кВт·год/добу та 27,60 кВт·год за 10 днів

6 Джерела даних, Room та транзакції

Головним постійним джерелом є Room Database energy_forecast.db. Додаткове локальне джерело AssetDemoScenarioSource читає JSON із assets/demo_scenarios.json. CachedDemoScenarioSource зберігає останнє успішне читання в оперативній пам’яті та синхронізує конкурентні звернення через Mutex. refresh() повторно читає джерело; якщо читання помилкове, попередній кеш не перезаписується. Віддаленого API та дискового JSON-кешу немає. Сутності та зв’язки бази наведено в табл. 3.

RoomEnergyRepository імпортує позиції вибраного JSON-шаблону до поточного профілю через database.withTransaction. Читання й перевірка JSON відбуваються до початку транзакції, а перевірка максимальної кількості приладів і вставлення — атомарно. Новим позиціям призначаються нові UUID, тому повторний імпорт може створювати прилади з однаковими назвами. JSON-шаблони не є окремими користувацькими сценаріями з власним CRUD. Вибір локального прикладу для імпорту показано на рис. 8.

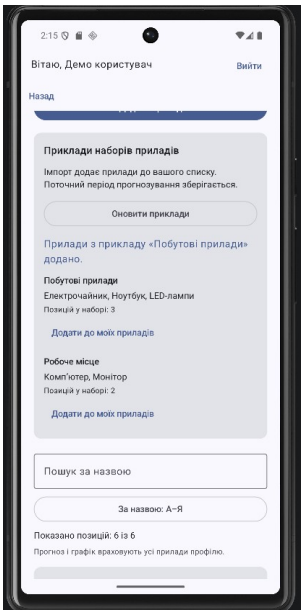


Рисунок 8 — Локальні JSON-приклади та імпорт приладів

Таблиця 3 — Сутності бази даних і зв’язки

Таблиця	**Призначення**	**Зв’язок**
energy_profiles	Період прогнозу для ownerKey	Первинний ownerKey
energy_devices	Поточні прилади	Складений ключ ownerKey + id; зовнішній ключ профілю
forecast_records	Назва, дата, дні та результати збереженого прогнозу	Ключ id, індекс ownerKey
forecast_items	Копії параметрів приладів на час збереження	Ключ forecastId + position; FK до forecast_records з ON DELETE CASCADE

Добове споживання поточного збереженого профілю обчислюється безпосередньо в EnergyDao.observeProfile(). Корельований підзапит охоплює лише прилади того самого ownerKey; COALESCE повертає 0.0 для профілю без приладів. Скорочений фрагмент запиту:

```
SELECT energy_profiles.*,
       (SELECT COALESCE(SUM(
           powerWatts / 1000.0 * hoursPerDay * deviceCount
       ), 0.0)
        FROM energy_devices
        WHERE energy_devices.ownerKey = energy_profiles.ownerKey
       ) AS dailyConsumptionKwh
FROM energy_profiles
WHERE energy_profiles.ownerKey = :ownerKey
```

Анотація @Transaction забезпечує узгоджене читання профілю, обчисленого поля й пов’язаного списку @Relation. Значення передається через RoomEnergyRepository → SavedEnergyProfile → ForecastViewModel → ForecastUiState.result до ForecastResultCard. Метод EnergyCalculator.calculateFromDaily() відхиляє від’ємний чи нескінченний підсумок, нечислові значення та переповнення добутку. dailyConsumptionKwh є полем результату запиту, а не новим стовпцем таблиці, тому версію бази не підвищено.

Операція ForecastHistoryDao.saveForecast() позначена @Transaction: спочатку записується forecast_records, потім усі forecast_items. Помилка другого кроку скасовує всю транзакцію. Історичні позиції мають власні значення назви, потужності, годин і кількості та не посилаються на поточну energy_devices, завдяки чому їхній вміст лишається незмінним після редагування поточного списку.

У версії бази 2 додано дві таблиці історії. MIGRATION_1_2 створює їх та потрібні індекси без видалення профілів і приладів. Під час створення Room використано addMigrations(MIGRATION_1_2), exportSchema=true; репозиторій містить 1.json і 2.json. Оновлення з наявними даними перевірено вручну. Окремий автоматизований тест переходу 1 → 2 не входить до наведених результатів тестування.

7 Розрахунковий алгоритм і контрольний приклад

Для кожної позиції враховуються потужність одного приладу P_i у ватах, тривалість роботи t_i у годинах за добу та кількість q_i. Сумарне добове споживання та прогноз за D днів:

$$Едоб = \Sigma (P_i / 1000 \cdot t_i \cdot q_i); \qquad Еперіод = Едоб \cdot D.$$

Потужність задається у ватах, час роботи — у годинах за добу, період — у днях; результат виражено в кВт·год. Округлення застосовується для відображення. Метод оцінює споживання за сталого режиму використання приладів; він не є моделлю машинного навчання та не замінює показання лічильника. Вхідні дані й внесок кожної позиції наведено в табл. 4.

Для поточного профілю суму Едоб повертає SQL-запит Room, а Еперіод обчислюється за добовим підсумком і вибраним D. Швидкий прогноз та збереження незалежної копії для історії використовують наявні методи EnergyCalculator за параметрами приладів. У всіх випадках застосовуються ті самі одиниці й формули.

Таблиця 4 — Контрольний розрахунок для шести позицій

Прилад	Потужність, Вт	Год/добу	К-сть	кВт·год/добу
Електрочайник	2000	0,25	1	0,50
Ноутбук	80	8	1	0,64
LED-лампи	10	5	6	0,30
Електрочайник	2000	0,25	1	0,50
Ноутбук	65	8	1	0,52
LED-лампи	10	5	6	0,30
РАЗОМ	—	—	6 позицій	2,76

Контрольний набір містить шість окремих записів: два електрочайники, два ноутбуки й дві позиції LED-ламп. Однакові назви не означають один запис: кожна позиція має власний ідентифікатор; ноутбуки відрізняються потужністю — 80 і 65 Вт. Кожна позиція ламп охоплює шість приладів, тому загальна кількість фізичних приладів дорівнює 16. За 10 днів: 2,76 × 10 = 27,60 кВт·год. На рис. 7 показано цей результат і фрагменти відповідного списку.

8 Історія збережених прогнозів

Під час збереження користувач задає назву довжиною 1–60 символів. RoomForecastHistoryRepository перевіряє копію списку від 1 до 50 позицій, перераховує результат саме для цієї копії, фіксує час і записує дані в дві пов’язані таблиці. У списку історії можна шукати за назвою прогнозу чи назвою приладу, обрати фільтр «Усі періоди / До 7 днів / Понад 7 днів» і сортування «Спочатку нові / За споживанням за період».

Список читається з Room, але його пошук, фільтрація і сортування виконуються у Kotlin у властивості ForecastHistoryUiState.visibleForecasts. Деталі включають збережені результати й перелік позицій; видалення запису відбувається після підтвердження в інтерфейсі, а каскадний зовнішній ключ видаляє пов’язані forecast_items. Список історії та деталі збереженого прогнозу наведено на рис. 9–10.

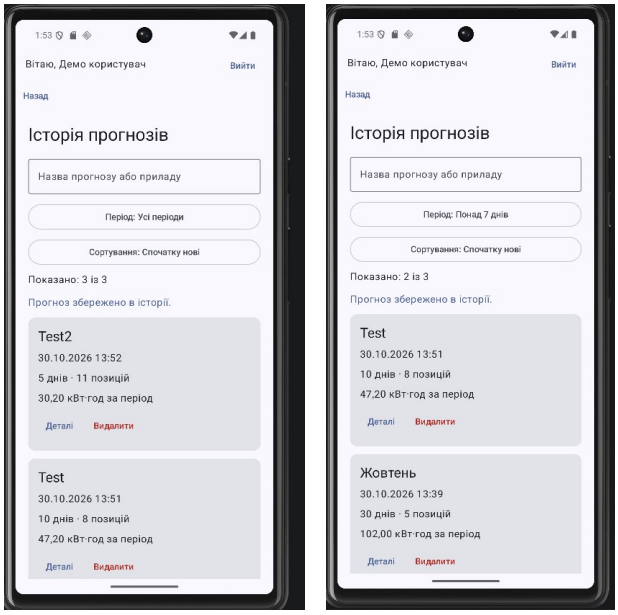


Рисунок 9 — Історія прогнозів і фільтрація за періодом

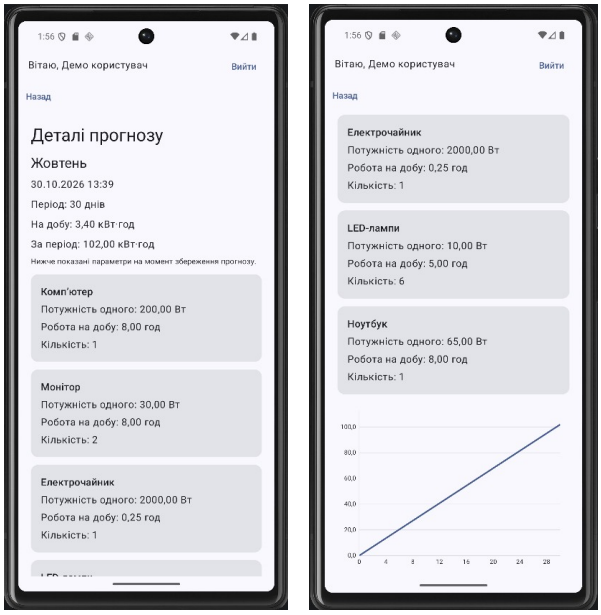


Рисунок 10 — Деталі збереженого прогнозу та зафіксовані параметри приладів

9 Адаптивність і перевірки

Інтерфейс використовує Material 3, прокручування, обмеження максимальної ширини та врахування появи клавіатури. Ручна перевірка охоплювала темну тему, альбомну орієнтацію, широкий екран, великий шрифт і відновлення діалогів. Приклади адаптивного вигляду наведено на рис. 11.

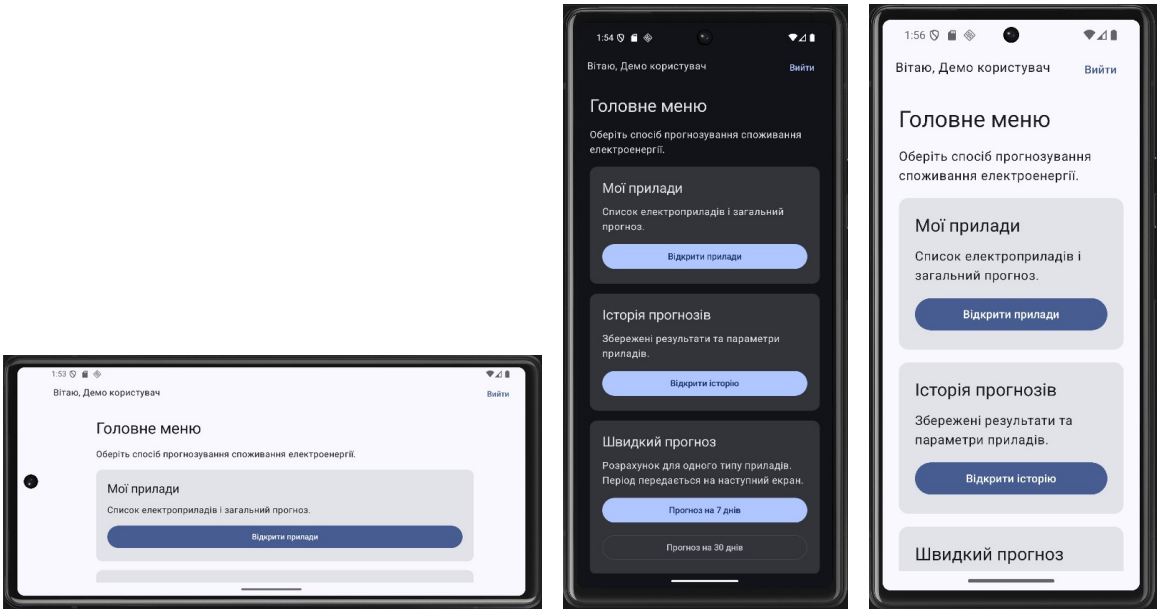


Рисунок 11 — Адаптивність інтерфейсу та темна тема

Для збирання та перевірки застосунку в PowerShell виконано такі команди:

```
.\gradlew.bat :app:testDebugUnitTest :app:lintDebug :app:assembleDebug --console=plain
.\gradlew.bat :app:connectedDebugAndroidTest --console=plain
.\gradlew.bat :app:installDebug --console=plain
```

Після виправлень команда :app:testDebugUnitTest :app:lintDebug :app:assembleDebug завершилася успішно для версії [2]. Інструментальні тести та встановлення на Pixel_6 (Android 16) також завершилися успішно. Після інструментального запуску змінено лише підготовку двох локальних тестів; код застосунку не змінювався. Локальні й інструментальні перевірки охоплюють логіку та Repository/Room; вони не є автоматизованими UI-тестами. Підсумки й джерела підтвердження наведено в табл. 5.

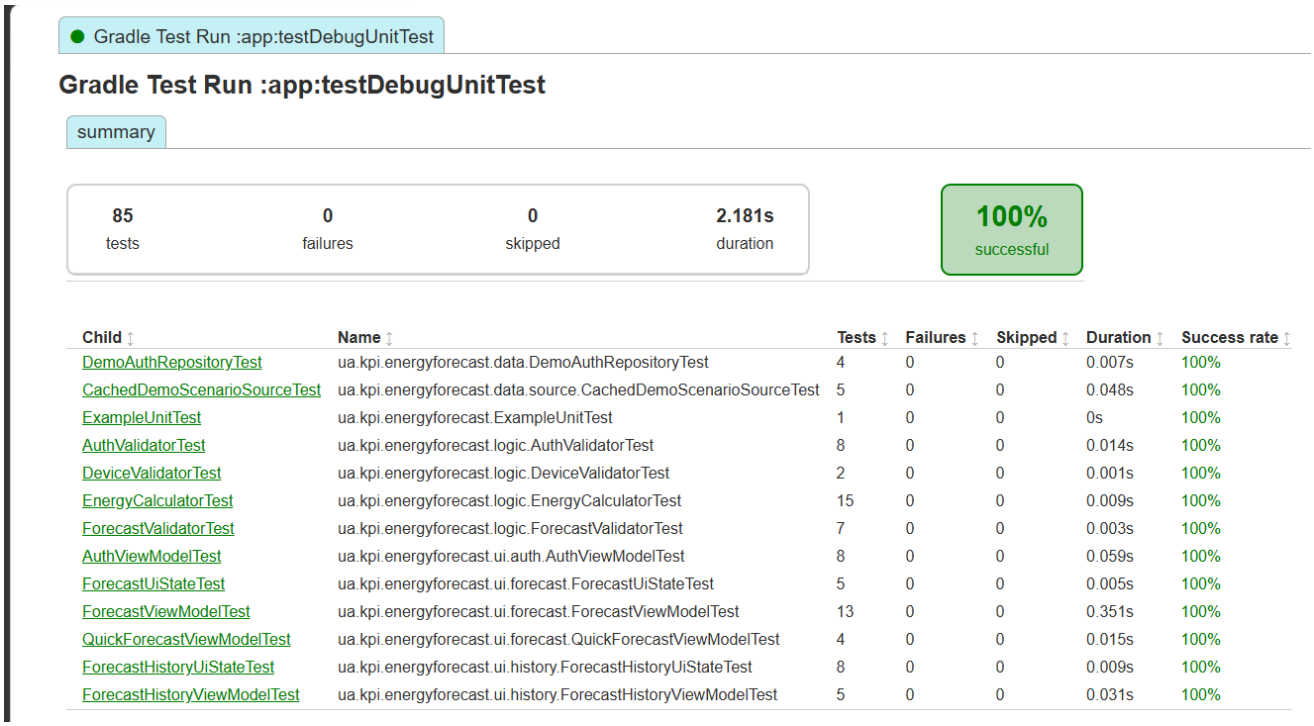
Таблиця 5 — Докази тестування та межі їх підтвердження

Перевірка	**Фактичний результат**	**Джерело / межі підтвердження**
Локальні unit-тести	85 виконано; 0 failures; 0 skipped; 100%; 2,181 с	Скриншот HTML-звіту Gradle, рис. 12
Інструментальні тести	14 виконано; 0 failures; 0 skipped; 100%; 1,429 с	HTML-підсумок, рис. 13; 13 тестів Repository/Room і 1 тест контексту; не UI-тести
Збірка та встановлення	assembleDebug, installDebug — успішні запуски	Результати локального запуску Gradle; підтвердження встановлення на Pixel_6
Ручні сценарії	Збереження, перезапуск, фільтри, діалоги, поворот	Ручна перевірка; приклади станів інтерфейсу — рис. 1–11
Android Lint	32 попередження; аналіз завершено успішно	Актуальний підсумок, рис. 14; кількість і категорії відповідають наведеному запуску

На рис. 12 підтверджено 85 локальних тестів: 0 failures, 0 skipped, 100% успіху; тривалість — 2,181 с. До них входить один стандартний ExampleUnitTest. На рис. 13 підтверджено 14 інструментальних тестів: 0 failures, 0 skipped, 100%; тривалість — 1,429 с. Із них 13 перевіряють Repository/Room, один — контекст застосунку.

Додані перевірки охоплюють використання добового підсумку зі сховища у ViewModel, зміну періоду, реакцію на нове значення підсумку, допустимі межі та переповнення. Інструментальні тести перевіряють нуль для порожнього профілю, підсумовування кількості приладів, оновлення Flow після додавання, редагування й видалення, а також ізоляцію ownerKey. Числовий набір одного з SQL-тестів є окремою тестовою фікстурою й не підмінює вхідні дані табл. 4.

Актуальний HTML-підсумок Android Lint (рис. 14) містить 32 попередження: AndroidGradlePluginVersion — 1, GradleDependency — 13, NewerVersionAvailable — 2, UnusedResources — 7 і UseTomlInstead — 9. Дубль android:label у Activity видалено; попередження RedundantLabel у цьому підсумку відсутнє. Зауваження стосуються версій інструментів і залежностей, невикористаних ресурсів та організації каталогу версій TOML. Аналіз завершився успішно, однак попередження залишаються. Їх не слід ототожнювати з невдалими тестами. Повідомлення sun.misc.Unsafe від protobuf-java належить до інструментів запуску, а не до перевірок бізнес-логіки.



Gradle Test Run :app:testDebugUnitTest

Gradle Test Run :app:testDebugUnitTest

summary

85	0	0	2.181s	100%
tests	failures	skipped	duration	successful

Child	Name	Tests	Failures	Skipped	Duration	Success rate
DemoAuthRepositoryTest	ua.kpi.energyforecast.data.DemoAuthRepositoryTest	4	0	0	0.007s	100%
CachedDemoScenarioSourceTest	ua.kpi.energyforecast.data.source.CachedDemoScenarioSourceTest	5	0	0	0.048s	100%
ExampleUnitTest	ua.kpi.energyforecast.ExampleUnitTest	1	0	0	0s	100%
AuthValidatorTest	ua.kpi.energyforecast.logic.AuthValidatorTest	8	0	0	0.014s	100%
DeviceValidatorTest	ua.kpi.energyforecast.logic.DeviceValidatorTest	2	0	0	0.001s	100%
EnergyCalculatorTest	ua.kpi.energyforecast.logic.EnergyCalculatorTest	15	0	0	0.009s	100%
ForecastValidatorTest	ua.kpi.energyforecast.logic.ForecastValidatorTest	7	0	0	0.003s	100%
AuthViewModelTest	ua.kpi.energyforecast.ui.auth.AuthViewModelTest	8	0	0	0.059s	100%
ForecastUiStateTest	ua.kpi.energyforecast.ui.forecast.ForecastUiStateTest	5	0	0	0.005s	100%
ForecastViewModelTest	ua.kpi.energyforecast.ui.forecast.ForecastViewModelTest	13	0	0	0.351s	100%
QuickForecastViewModelTest	ua.kpi.energyforecast.ui.forecast.QuickForecastViewModelTest	4	0	0	0.015s	100%
ForecastHistoryUiStateTest	ua.kpi.energyforecast.ui.history.ForecastHistoryUiStateTest	8	0	0	0.009s	100%
ForecastHistoryViewModelTest	ua.kpi.energyforecast.ui.history.ForecastHistoryViewModelTest	5	0	0	0.031s	100%

Рисунок 12 — Підсумок локальних unit-тестів Gradle: 85 тестів, 0 помилок, 0 пропусків

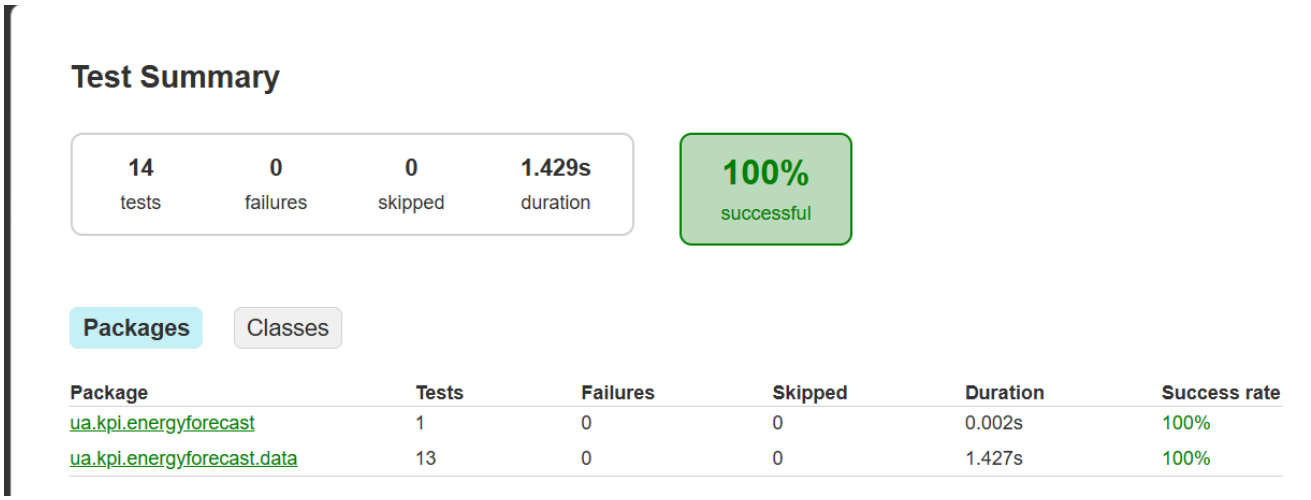


Рисунок 13 — Підсумок інструментальних тестів: 14 тестів, 0 помилок, 0 пропусків

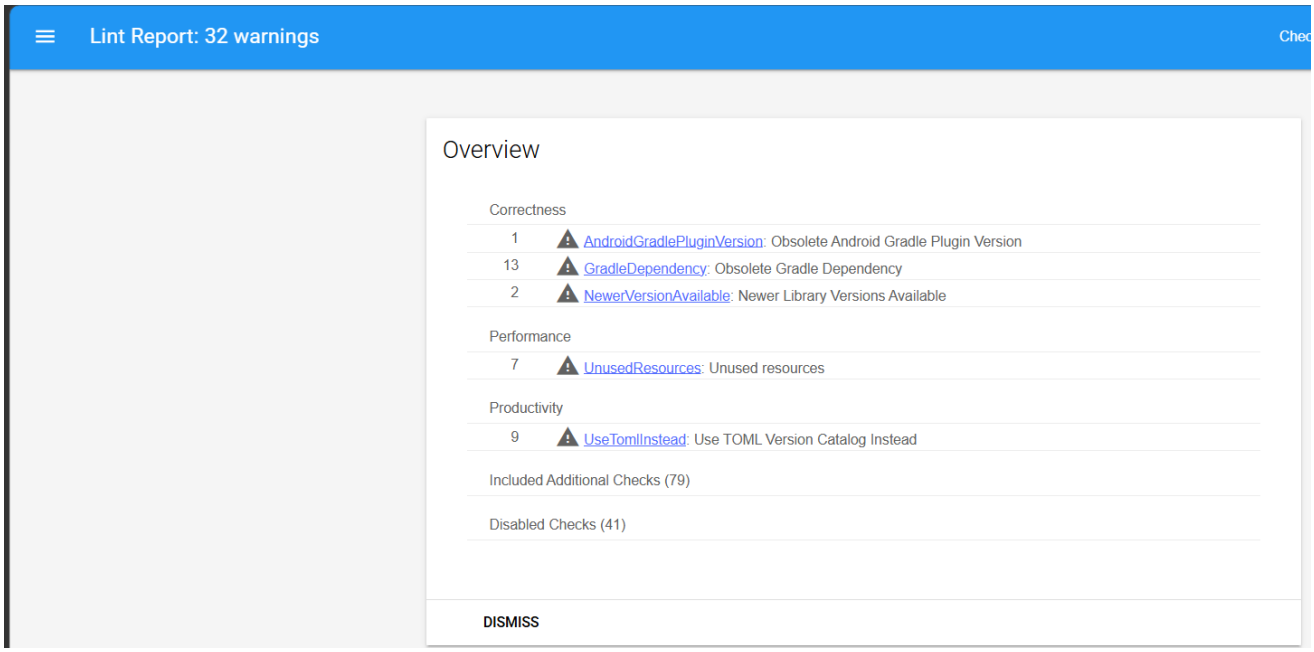


Рисунок 14 — Актуальний звіт Android Lint: 32 попередження за категоріями

10 Обмеження та висновки

Застосунок EnergyForecast реалізує керування електроприладами, розрахунок споживання, імпорт локальних прикладів і збереження історії прогнозів. Navigation Compose поєднує екрани, MVVM відокремлює стан і логіку від інтерфейсу, а Room забезпечує постійне збереження даних та транзакційний запис пов’язаних сутностей.

Обмеження реалізації: авторизація демонстраційна, без сервера й постійного сховища облікових записів; прив’язка Room до нормалізованого email не забезпечує захисту даних. Розрахунок припускає незмінні параметри приладів щодня. Пошук історії виконується в пам’яті, а JSON-кеш зникає після завершення процесу. Окремий автоматизований тест міграції не наведено. Спільний стан повідомлень історії може показувати повідомлення про видалення біля картки збереження.

У роботі реалізовано меню з навігаційними аргументами, форми з валідацією, реактивний стан і локальне сховище для варіанта №1. Робочий SQL-запит обчислює добовий підсумок, який використовується в картці прогнозу. Контрольний розрахунок узгоджується з 2,76 кВт·год/добу та 27,60 кВт·год за 10 днів. Надані підсумки підтверджують успішне проходження 85 локальних і 14 інструментальних тестів без помилок та пропусків. Lint завершився з 32 попередженнями, які проаналізовано окремо.

Додаткові навчальні джерела з мови Kotlin: інтерактивні вправи Kotlin Koans [5] та посібник Atomic Kotlin [6].

Список використаних джерел

[1] Розробка програмного забезпечення мобільних пристроїв. Практична робота №2 (практичні заняття 5 –8): методичні вказівки. КПІ ім. Ігоря Сікорського, Київ, 2026. 192 с.

[2] RPZMP, каталог PW02, commit feb433757ed9896859ba673c6d69fca10fec9021. https://github.com/Christ1naa/RPZMP/tree/feb433757ed9896859ba673c6d69fca10fec9021/PW02

[3] Android Developers. Jetpack Compose, Navigation, ViewModel і Room [Електронний ресурс]. — Режим доступу: https://developer.android.com/. — Назва з екрана.

[4] Kotlin docs [Електронний ресурс]. — Режим доступу: https://kotlinlang.org/docs/home.html. — Назва з екрана.

[5] Kotlin Koans [Електронний ресурс]. — Режим доступу: https://play.kotlinlang.org/koans/overview. — Назва з екрана.

[6] Eckel B., Isakova S. Atomic Kotlin [Електронний ресурс]. — Режим доступу: https://www.atomickotlin.com/atomickotlin/. — Назва з екрана.